DBMS ASSIGNMENT

ASSIGNMENT 4

Relational Database vs. NoSQL Database

Project: Library Loan Management System

Database: MySQL 8.0

Objective

Compare relational and NoSQL databases and justify a suitable technology for the Library Loan Management System.

Note: outputs are expected results based on the included sample data. Run the SQL in your MySQL environment and insert your own screenshots if required by your instructor.

1. Introduction

Database technology should be chosen based on the structure of data, consistency requirements, query patterns, scalability, security, and operational cost. A library system has well-defined entities and relationships.

2. Relational Databases

A relational database stores data in tables and uses primary keys, foreign keys, constraints, and SQL. Examples include MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server. Relational databases are well suited to transactions and reports joining related entities.

3. NoSQL Databases

NoSQL is a broad category including document databases such as MongoDB, key-value stores such as Redis, wide-column databases such as Cassandra, and graph databases such as Neo4j. Their data models and consistency guarantees vary by product. NoSQL does not automatically mean that transactions or consistency are unavailable.

4. Comparison

Data model: relational systems use tables and defined schemas; document databases can store nested, flexible documents.
Relationships: relational databases support joins and foreign keys; NoSQL systems may embed related data, use references, or provide model-specific graph traversal.
Consistency: relational products commonly provide strong transaction guarantees; NoSQL capabilities depend on product and configuration.
Scalability: both categories can scale; some NoSQL systems are designed for distributed horizontal scaling, while modern relational systems also support replication and partitioning.
Queries: SQL is widely used for relational data; NoSQL query methods vary by product.
Integrity: relational constraints can centrally enforce member, copy, and loan rules. Some NoSQL designs require more validation in application code.
Performance: depends on workload, indexes, data model, hardware, and tuning—not only the database category.

5. Best Fit for the Library Project

MySQL 8.0 is recommended for the core Library Loan Management System. Members, books, authors, copies, and loans have clear relationships. Foreign keys protect references, unique constraints prevent duplicate accession numbers and simultaneous open loans, and SQL joins support overdue and popularity reports. A document database might be useful for flexible user reviews or activity logs, but it is not necessary for the core project.

6. Real-World Use Cases

Relational: fee ledgers, banking transactions, library circulation, inventory with strict references.
Document: book descriptions with variable metadata or user-generated reviews.
Key-value: caching popular catalogue searches or login sessions.
Graph: exploring author collaborations and citation networks.
The technology should follow the required workload rather than popularity alone.

7. Selection Framework

Before choosing a production database, evaluate data structure, transaction boundaries, consistency needs, expected growth, query patterns, security, backup/recovery, team skills, hosting cost, and migration effort. Test representative workloads before deployment.

8. Conclusion

RDBMS and NoSQL databases have different strengths. A relational database is the clearest choice for this library system because it requires structured records, reliable relationships, and multi-table reporting.